INSTITUTO POLITÉCNICO NACIONAL · ESCUELA SUPERIOR DE CÓMPUTO

Cambio de esquema por integrante Bitácora de carga y auditoría de registros descartados

Unidad de aprendizaje	Bases de Datos
Práctica	Práctica 3 — Ejercicio 8
Integrante	Mora Acosta Pablo — `pablo140706`
Proyecto asignado	Sistema de visualización de datos sísmicos
Issue	pablo140706/Seismic-Data-Visualization-System#2
Base de trabajo	`dw_pablo140706`, copia de `datawarehouse`

El problema que atiende

El almacén se construye a partir de datos abiertos heterogéneos. Durante la carga se descartan filas: coordenadas fuera del territorio nacional, magnitudes ausentes, entidades que no casan con el catálogo del censo, fechas imposibles. Hoy ese descarte no deja rastro. La fila simplemente no llega a la tabla de hechos y nadie puede saber cuántas se perdieron, por qué motivo ni de qué archivo venían.

La consecuencia práctica es que no se puede distinguir «no hubo sismos ese mes» de «la carga falló a la mitad». Para un almacén cuyo propósito es analizar impacto, esa ambigüedad contamina cualquier conclusión: un estado puede parecer poco afectado simplemente porque sus registros se cayeron en la validación.

Cambios en el esquema

Dos tablas nuevas y una columna añadida a la tabla de hechos.

Tabla 1. Tablas nuevas.

Tabla	Qué guarda	Clave primaria
`carga_lote`	Una fila por ejecución del proceso de carga: archivo de origen, tabla destino, inicio y fin, filas leídas, insertadas y descartadas, y el estado	`id_lote`
`registro_descartado`	Una fila por registro rechazado: el lote al que pertenece, el número de línea, el motivo, el detalle y el contenido original	`id_descarte`

Y sobre la tabla de hechos:

```
ALTER TABLE fact_impacto_sismos_imputed ADD COLUMN id_lote INTEGER;
ALTER TABLE fact_impacto_sismos_imputed
    ADD CONSTRAINT fk_fact_lote FOREIGN KEY (id_lote)
        REFERENCES carga_lote (id_lote) ON DELETE SET NULL;
```

Reglas de transformación aplicadas

Regla 1 para las dos entidades nuevas: cada una se convierte en una tabla, con una columna por atributo simple y su identificador como clave primaria. Se añadieron las restricciones que el dominio exige: el estado solo admite `en_proceso`, `completado` o `fallido`; los conteos no pueden ser negativos; y la suma de insertadas y descartadas no puede superar a las leídas, que es la invariante de cualquier proceso de carga.

Regla 4 para la relación entre ambas. Un lote descarta cero o muchos registros; cada registro descartado pertenece a exactamente un lote. La cardinalidad es `(0,N) : (1,1)`, así que la clave del lado uno se propaga al lado muchos como clave foránea, y como la cardinalidad mínima del lado muchos es 1, esa foránea es `NOT NULL`. Lleva `ON DELETE CASCADE`: un registro descartado no describe nada sin la carga que lo produjo.

La columna añadida a la tabla de hechos es también Regla 4, pero con una diferencia importante: **admite nulos**, porque las filas que ya existen se cargaron antes de que existiera la bitácora y no se les puede inventar un lote de origen.

Por qué esos tipos de dato

Tabla 2. Decisiones de tipo y su justificación.

Columna	Tipo	Por qué
`inicio`, `fin`, `registrado_en`	`TIMESTAMPTZ`	Con zona horaria y no `TIMESTAMP` a secas, porque los datos sísmicos vienen en UTC —la propia `dim_tiempo` guarda `hora_utc`— y el servidor puede estar en otra zona. Sin la zona, dos cargas idénticas hechas desde equipos distintos quedarían registradas en momentos aparentemente diferentes
`contenido`	`JSONB`	Es la decisión de fondo. Las fuentes tienen columnas distintas entre sí: el archivo del SSN no se parece al del censo. Guardar la fila original como `TEXT` obligaría a volver a parsearla para entender qué falló; con `JSONB` se consulta campo por campo — `contenido->>'latitud'` — y el gestor la almacena ya descompuesta e indexable. Se eligió `JSONB` y no `JSON` porque el segundo guarda el texto literal y vuelve a analizarlo en cada consulta
`motivo`	`VARCHAR(40)` con `CHECK`	Un catálogo cerrado de seis motivos, no

		texto libre. Si cada quien escribe el motivo a mano, la consulta que agrupa por motivo deja de servir
`numero_linea`	`INTEGER` con `UNIQUE (id_lote, numero_linea)`	Permite volver al archivo original y localizar la fila exacta. El `UNIQUE` impide registrar dos veces el mismo descarte si el proceso se reintenta

La consulta que solo es posible después del cambio

Antes del cambio esta pregunta no tenía respuesta, porque lo descartado no se guardaba en ningún lado.

```
SELECT l.id_lote, l.archivo_origen, l.tabla_destino, d.motivo,
       COUNT(*) AS descartados,
       ROUND(100.0 * COUNT(*) / l.filas_leidas, 2) AS pct_del_archivo
  FROM carga_lote l
  JOIN registro_descartado d ON d.id_lote = l.id_lote
 GROUP BY l.id_lote, l.archivo_origen, l.tabla_destino, l.filas_leidas, d.motivo
 ORDER BY l.id_lote, descartados DESC;
```

Y una segunda, que es la que da valor operativo: recuperar las filas rechazadas para corregirlas y volver a cargarlas.

```
SELECT d.id_lote, d.numero_linea,
       d.contenido ->> 'fecha'    AS fecha,
       d.contenido ->> 'latitud'  AS latitud,
       d.contenido ->> 'longitud' AS longitud,
       d.detalle
  FROM registro_descartado d
 WHERE d.motivo = 'coordenada_invalida'
 ORDER BY d.id_lote, d.numero_linea;
```

Ejecución

```
docker exec -i seismic-data-visualization-system-db-1 \
    psql -U postgres -d dw_pablo140706 -v ON_ERROR_STOP=1 \
    < cambios/cambio-pablo140706.sql
```

```
PS C:\Users\0\Documents\escuela\base_de_datos\practica3-referencia> Get-Content cambios\PABLO--cambio-pablo140706.sql | docker exe
c -i seismic-data-visualization-system-db-1 psql -U postgres -d dw_pablo140706 -v ON_ERROR_STOP=1
BEGIN
CREATE TABLE
COMMENT
CREATE TABLE
COMMENT
CREATE INDEX
CREATE INDEX
ALTER TABLE
ALTER TABLE
COMMIT
BEGIN
INSERT 0 3
INSERT 0 7
UPDATE 3
COMMIT
=== Auditoria de cargas: que se perdio y por que ===
 id_lote |   archivo_origen    | tabla_destino |       motivo         | descartados | pct_del_archivo
---------+---------------------+---------------+----------------------+-------------+-----------------
       1 | sismos_ssn_2024.csv | dim_sismos    | coordenada_invalida  |           2 |            0.17
       1 | sismos_ssn_2024.csv | dim_sismos    | fecha_invalida       |           1 |            0.08
       1 | sismos_ssn_2024.csv | dim_sismos    | duplicado            |           1 |            0.08
       1 | sismos_ssn_2024.csv | dim_sismos    | magnitud_ausente     |           1 |            0.08
       3 | economia_inegi.csv  | dim_economia  | entidad_no_catalogada |          2 |            5.00
(5 rows)


=== Las filas con coordenada invalida, listas para corregir ===
 id_lote | numero_linea |   fecha    | latitud | longitud |             detalle
---------+--------------+------------+---------+----------+---------------------------------
       1 |           54 | 2024-03-11 | 17.9    | -204.5   | Longitud fuera del rango de Mexico
       1 |          310 | 2024-05-21 | 99.9    | -95.2    | Latitud fuera del rango de Mexico
(2 rows)
```

Qué ocurre con los datos que ya existen

El cambio es seguro de aplicar sobre una base en producción, y conviene poder explicar por qué.

Las dos tablas nuevas nacen vacías y no tocan nada de lo cargado: ninguna fila existente se modifica ni se borra. La columna `id_lote` que se añade a la tabla de hechos admite nulos, así que las filas que ya estaban quedan con `NULL` y el `ALTER TABLE` no falla. En PostgreSQL, añadir una columna que admite nulos y sin valor por defecto no reescribe la tabla, de modo que la operación es prácticamente instantánea aun con la tabla llena.

El precio es que **no hay bitácora retroactiva**. De las cargas anteriores a este cambio no se sabe —ni se puede saber— cuántas filas se descartaron: esa información se perdió en su momento y no está en ningún lado del que recuperarla. La auditoría empieza a contar desde la siguiente carga. Si el equipo quisiera histórico, habría que volver a procesar los archivos originales con el proceso de carga ya instrumentado, lo que es posible pero excede este cambio.

Una última consideración: el `ON DELETE SET NULL` de la tabla de hechos es deliberado. Si alguien borrara un lote de la bitácora, las filas de hechos cargadas por él deben sobrevivir y simplemente quedarse sin trazabilidad; perder datos de impacto sísmico por limpiar una bitácora sería mucho peor que perder la trazabilidad.

Referencias

Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

The PostgreSQL Global Development Group. (2025). *PostgreSQL 16 documentation: JSON types and ALTER TABLE*. https://www.postgresql.org/docs/16/

Villa Vargas, J. M., Hurtado Avilés, G., y Climent Hernández, J. A. (2026). Cuando México tiembla: la historia contada por los datos. *AZCATL Revista de Divulgación de la Ciencia, la Ingeniería y la Innovación, 4*(6), 28–33. https://doi.org/10.24275/AZC2026E1004